

Guía rápida de repaso EDA: árboles, grafos y Java de examen

1. Mentalidad de examen

Antes de programar, identifica:

- qué estructura gobierna el problema;
- cuál es la clave de búsqueda;
- qué índices secundarios necesitas mantener;
- qué invariante no puedes romper;
- qué casos límite hay: vacío, un elemento, duplicados, null, raíz, hoja, grafo desconectado.

En implementación pura no sustituyas la estructura pedida por otra. En casos de uso, sí debes elegir bien entre HashMap, HashSet, TreeMap, TreeSet y PriorityQueue.

2. Java usado en la asignatura

Patrón típico:

```
public static <V, E> boolean metodo(AdjacencyMapGraph<V, E> graph, Vertex<V> v) {
    if (graph == null || v == null) {
        throw new IllegalArgumentException();
    }
    // TODO
}
```

Ideas clave:

- usa genéricos (`<`, `>`, ``);
- respeta firmas públicas;
- usa `Position` para árboles;
- usa `Vertex` y `Edge` para grafos;
- usa `Comparator` cuando el orden no es natural;
- `UnsupportedOperationException` es correcto en esqueletos, no en soluciones finales.

3. Árboles generales

Un árbol se razona siempre desde:

- raíz;
- padre;
- hijos;
- hojas;
- profundidad;
- altura;
- grado.

Recorrido recursivo típico:

```
private static <E> int size(Tree<E> t, Position<E> p) {
    int total = 1;
    for (Position<E> child : t.children(p)) {
```

```
total += size(t, child);
}
return total;
}
```

BFS por niveles:

```
Queue<Position<E>> q = new LinkedList<>();
q.add(t.root());
while (!q.isEmpty()) {
    Position<E> p = q.poll();
    for (Position<E> child : t.children(p)) {
        q.add(child);
    }
}
```

4. Árboles binarios

En binarios piensa en:

- hijo izquierdo;
- hijo derecho;
- nodo con 0, 1 o 2 hijos;
- subárbol izquierdo;
- subárbol derecho.

Patrones:

- ``isSymmetric``: comparar espejo izquierdo/derecho.
- ``areIdentical``: comparar elemento y estructura a la vez.
- ``hasSameShape``: comparar solo estructura.
- ``isBalanced``: cada nodo necesita alturas de hijos.
- ``diameter``: máximo entre diámetro izquierdo, derecho y camino que pasa por la raíz.

Para montículos:

1. estructura casi completa;
2. orden padre-hijo según comparador.

Si falla cualquiera, no es heap.

5. Iteradores de árbol

Un iterador necesita estado:

- pendientes por visitar;
- último devuelto;
- si ``remove()`` puede ejecutarse;
- qué pasa si se borra un subárbol.

Errores típicos:

- visitar nodos que ya no están en el árbol;
- permitir dos ``remove()`` seguidos;
- no lanzar ``NoSuchElementException`` cuando no hay siguiente;
- cambiar el árbol y no actualizar la cola/pila interna.

6. Grafos del entorno

En AdjacencyMapGraph piensa en:

- ``vertices()``;
- ``edges()``;
- ``opposite(v, e)``;
- ``incidentEdges(v)``;
- ``areAdjacent(u, v)``;
- ``insertVertex``;
- ``insertEdge``.

BFS base:

```
HashSet<Vertex<V>> visited = new HashSet<>();
HashMap<Vertex<V>, Integer> dist = new HashMap<>();
Queue<Vertex<V>> q = new LinkedList<>();
visited.add(start);
dist.put(start, 0);
q.add(start);
while (!q.isEmpty()) {
    Vertex<V> u = q.poll();
    for (Edge<E> e : graph.incidentEdges(u)) {
        Vertex<V> w = graph.opposite(u, e);
        if (!visited.contains(w)) {
            visited.add(w);
            dist.put(w, dist.get(u) + 1);
            q.add(w);
        }
    }
}
```

Este patrón sirve para:

- distancia mínima;
- vértices a distancia k;
- componentes conexas;
- diámetro;
- centro;
- bipartito;
- caminos mínimos.

7. Grafos con HashMap y HashSet en casos de uso

Grafo no dirigido:

```
HashMap<String, HashSet<String>> adj = new HashMap<>();

adj.get(a).add(b);
adj.get(b).add(a);
```

Grafo dirigido:

```
HashMap<String, HashSet<String>> salientes = new HashMap<>();
HashMap<String, HashSet<String>> entrantes = new HashMap<>();
```

BFS manual:

```
HashSet<String> visitados = new HashSet<>();
Queue<String> cola = new LinkedList<>();
```

```
visitados.add(origen);  
cola.add(origen);
```

8. Estructuras permitidas para casos de uso

Trampa clave: TreeSet y PriorityQueue no se reordenan mágicamente si cambias el campo usado para ordenar. Quita y vuelve a insertar.

9. Checklist antes de entregar

- ¿Compila?
- ¿He mantenido firmas públicas?
- ¿He probado vacío y un elemento?
- ¿He probado duplicados?
- ¿He probado grafo desconectado?
- ¿He probado raíz y hojas?
- ¿He actualizado todos los índices?
- ¿He quitado y reinsertado en `TreeSet`/`PriorityQueue` si cambia la prioridad?
- ¿Puedo explicar la complejidad?